

```

return tokenizer.decode(
    output[0, inputs.input_ids.shape[1]:],
    skip_special_tokens=True,
)

test_prompt = (
    «### Instruction:\n»
    «Demonstrate pthread condition variables with a producer/
consumer of a single-slot buffer.\n\n»
    «### Response:\n»
)

print(generate(test_prompt))

```

PeftModel.from_pretrained(base_model, LORA_DIR, is_trainable=False): This attaches LoRA adapters learned during fine-tuning on top of the frozen, quantized backbone—yielding parameter-efficient domain adaptation with minimal memory/latency overhead.

model.eval(): This disables dropout; `use_cache=True` activates KV caching for fast autoregressive decoding; special token IDs are synchronised with the tokenizer to ensure clean termination and padding.

A helper `generate()` function tokenizes prompts on the model device, runs inference under `torch.no_grad()`, and uses stochastic decoding (temperature, top-p, top-k, repetition penalty) to balance diversity and coherence, returning only the newly generated tokens beyond the prompt.

Step 8: Merge and save the model

```

# Merge and save the model
merged_model = infer_model.merge_and_unload()
merged_model.save_pretrained(f"{OUTPUT_DIR}_merged")
tokenizer.save_pretrained(f"{OUTPUT_DIR}_merged")

```

```
print(f"Merged model saved to: {OUTPUT_DIR}_merged")
```

This stage finalises the fine-tuned model by merging LoRA adapter weights into the underlying quantized base model to produce a single, standalone checkpoint suitable for deployment. The call to `merge_and_unload()` collapses the learned low-rank LoRA updates into the original linear layers, removing the PEFT wrapper and eliminating the need for LoRA adapters at inference time. This yields a consolidated model with all fine-tuning deltas baked directly into its weights, improving runtime simplicity, compatibility, and portability across inference engines.

The merged model is then saved using `save_pretrained()`, along with the tokenizer, to a new directory. This directory contains a fully self-contained model artifact that can be served without requiring PEFT components, LoRA configuration files, or training scaffolding. The result is a clean, production-ready set of model files optimised for real-world deployment scenarios.

This is a practical, end-to-end blueprint to fine-tune Mistral-7B-Instruct into a C and Linux system-programming assistant using PEFT with QLoRA and 4-bit quantization on a Colab T4 GPU. The workflow is reproducible and emphasises memory-efficient, stable training to enable domain adaptation on constrained hardware. **END** 🐧

 By: Subranshu Patel and Shwetabh Srivastava

Subranshu Patel is a group manager in the system software division of HCL Technologies Limited. He has extensive experience in Linux-based OS and ecosystem design and development, and in open source technologies.

Shwetabh Srivastava is a solutions architect at HCL Technologies Limited. He is experienced in designing and developing Linux-based system software, with expertise in operating system principles and the inner workings of the Linux kernel.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU